

별무리

AI·SW 마에스트로 서울 제17기

2026.04 ~ 2026.11 (정식 출시 예정) / 온디바이스 AI 개발

소스 코드: <https://github.com/mornye-minor-gallery/PetAI-AI>

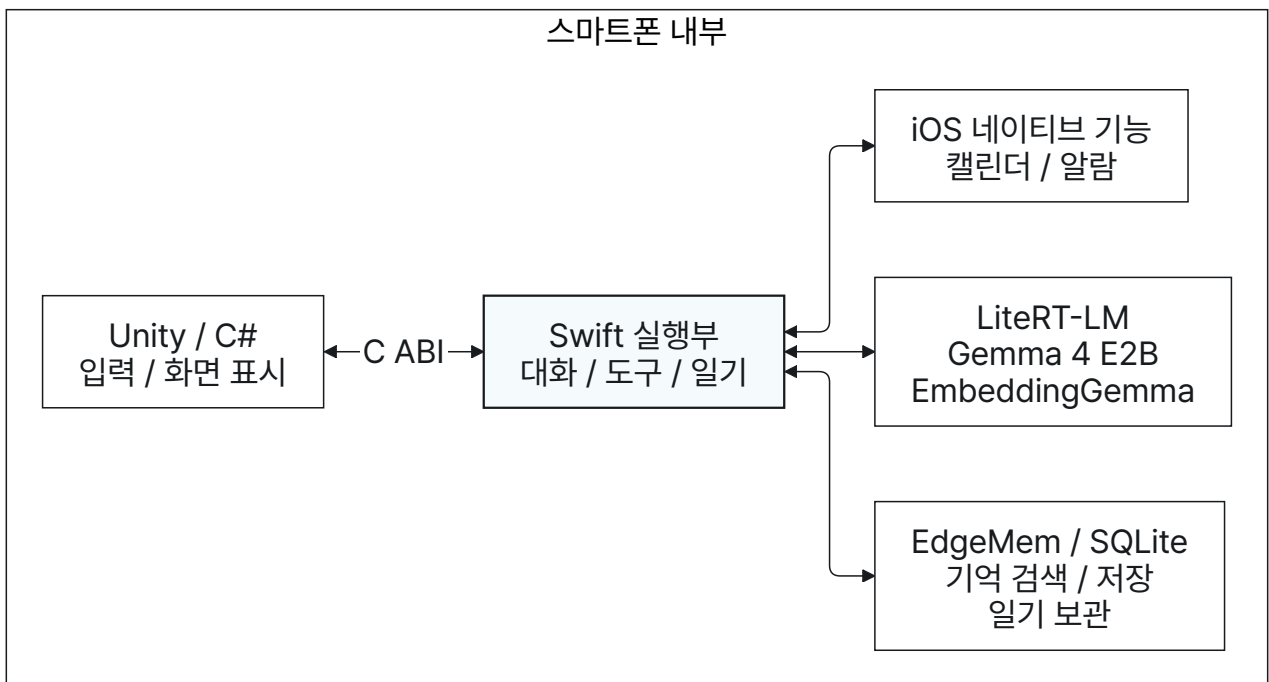
프로젝트 개요

- 별무리는 스마트폰에서 완전히 온디바이스로 실행되는 소형 언어 모델(sLM)을 통해 캐릭터와 대화하며 개인화된 경험을 제공하는 앱입니다. 그날의 중요한 기억을 매일 일기로 저장하며, 자연어로 캘린더 정리와 알람 설정 등의 네이티브 기능을 호출하여 작업하는 Agent 기능을 지원합니다.

담당 범위

- Effective Parameter 2.3B의 파라미터 규모를 가진 소형 언어 모델 Gemma 4 E2B 모델을 활용하여 완전한 모바일 (Android, iOS) 온디바이스 추론을 구현했습니다.
- KVCache 등의 최적화를 위해 Google의 오픈소스 엣지 디바이스 추론 엔진인 LiteRT-LM을 프로젝트에 맞게 직접 수정하고 적용했습니다.
- 300M의 파라미터 규모를 가진 EmbeddingGemma를 활용한, 개인화 소형 언어 모델을 위한 장기 기억 아키텍처 EdgeMem을 직접 설계하고 적용했으며, iOS 네이티브 앱 기능 Tool Calling을 구현했습니다.
- 게임 클라이언트 Unity와 LiteRT-LM 추론 엔진 실행부를 책임지는 Swift 엔진을 Application Binary Interface로 통신하여 C#과 C++ 두 언어의 동작을 통합했습니다.

전체 구조

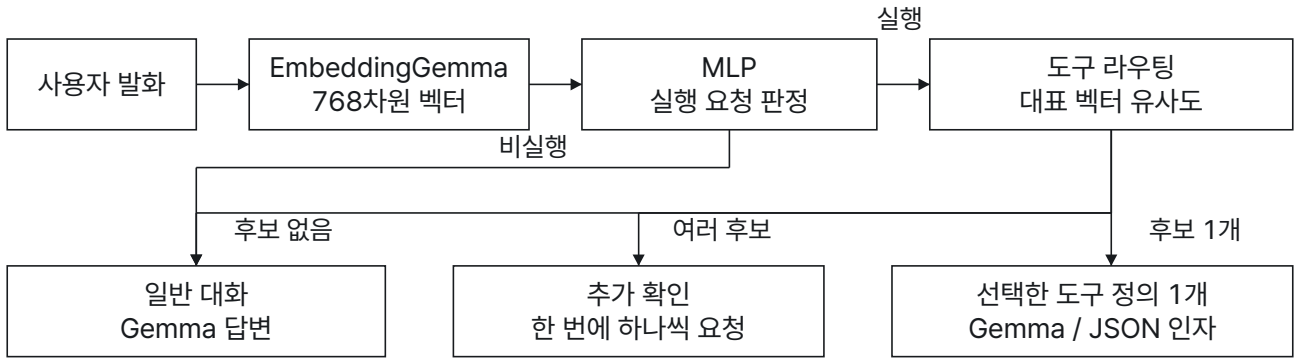


- Unity는 C ABI로 사용자 입력을 Swift 실행부에 전달하고, 답변과 실행 결과를 받아 화면에 표시합니다. Swift는 대화와 도구 요청을 구분하며, LiteRT-LM의 모델 실행, EdgeMem의 기억 검색과 저장, iOS 네이티브 기능 호출을 처리합니다. 일기는 그날 저장한 기억으로 생성해 기기에 보관합니다.

사용 기술

클라이언트	Unity / C#
네이티브 실행부와 연동	Swift / C/C++ / C ABI
추론 엔진	LiteRT-LM
모델	Gemma 4 E2B / EmbeddingGemma
기억 저장	SQLite
학습과 평가	Python

1. 일반 대화의 도구 호출 오인을 줄인 실행 의도 분류



문제

- 개발 중 테스트에서 일반 대화가 도구 요청으로 잘못 분류되는 경우를 반복해서 확인했습니다.
- "내가 만든 알람 목록을 확인하는 기능도 지원해?" 같은 기능 문의도 오인했습니다. 이런 까다로운 형태의 문장을 **Hard Negative**라고 합니다. Gemma에 직접 판정을 맡기더라도 이런 **False Positive**가 많았습니다 (일반 대화 24건 중 9건을 실행 요청으로 분류).
- 사용자가 원하지 않은 기능 처리로 넘어가는 상황은 불쾌감과 불편함이 크다고 생각했습니다.

해결 과정

- 키워드가 같아도 기능 문의와 실행 요청은 구분해야 했습니다. 기존 EmbeddingGemma의 발화 표현에 작은 분류 헤드를 학습하면 구분할 수 있을 것이라고 생각했습니다.
 - 768차원 벡터를 입력받는 MLP로 실행 의도를 판정하고, 통과한 발화의 같은 벡터로 도구를 선택했습니다.
- 소형 모델에 맡기는 작업을 줄이려고, Gemma에는 선택된 도구 하나의 정의만 전달해 JSON 인자를 생성하게 했습니다.
- 도구와 표현이 비슷하지만 실행 요청이 아닌 **Hard Negative**를 학습 데이터에 추가했습니다.
 - 한국어 **Hard Negative**만 보강하자 일반 대화 오인은 줄었지만 실제 실행 요청도 많이 놓치는 문제가 발생했습니다. 그래서 **Hard Negative** 샘플에 해당하는 비실행 문장을 LLM 후처리로 생성한 도구 실행 문장을 **Pair**로 사용하여 데이터를 증강하고 함께 보강했습니다.
- 테스트
 - 기존 평가셋의 실제 실행 요청 168건과 일반 대화 24건으로 학습 구성을 비교했습니다.

결과

학습 구성별 분류 결과

학습 구성	대화 오인율 ↓	요청 탐지율 ↑
초기 MLP	66.7%	97.0%
한국어 Hard Negative만 보강	8.3%	70.2%
실행/비실행 함께 보강	8.3%	95.2%
위 구성 + 한국어 Hard Negative	4.2%	94.6%

- 일반 대화의 도구 요청 오인을 임베딩 기준선 대비 **90.9%** 줄였습니다(**11/24건 → 1/24건**).
- 마지막 두 구성은 전체 정답 수가 같았습니다. 일반 대화 오인을 줄이는 것을 우선해, 실제 요청을 한 건 더 놓치더라도 오인이 적은 구성을 선택했습니다.

2. KV 캐시 재사용과 앱 재실행 후 복원

문제

- TestFlight 베타 사용자들이 공통적으로 답변이 늦게 나온다는 피드백을 주었습니다.
- 답변이 나오기 시작하면 출력 속도는 괜찮았지만, 시작까지 기다리는 시간이 길게 느껴졌습니다. 프리필이 느린 것 같았습니다.
- 앱 전용의 고정 프롬프트가 매 요청에 포함됐습니다. 캐시를 유지하지 않으면 같은 프롬프트를 반복해서 계산해야 했습니다.

해결 과정

- 처음에는 메모리의 KV 캐시를 재사용하는 기능만 구현하려고 했습니다.
 - CachedSession으로 메모리의 KV 캐시를 유지하고, 같은 고정 프롬프트를 다음 요청에 재사용했습니다.
- 앱을 종료하면 메모리의 캐시가 사라져, 재실행한 뒤에는 같은 프롬프트를 다시 계산해야 한다는 점을 떠올렸습니다.
 - 앱을 껐다 켜도 캐시를 재사용하려면, 추론 상태를 파일로 저장하고 새 세션에 복원해야 했습니다.
- **사용하던 LiteRT-LM의 공개 C API에는 추론 상태를 앱으로 내보내고 다시 복원하는 기능이 없었습니다.** LiteRT-LM을 포크하고 CPU 세션의 상태 전송 API를 추가했습니다.
 - 엔진에서 KV 텐서와 처리 위치, 토큰 ID와 샘플러 상태를 전송했습니다.
 - Swift에서 파일 저장과 복원, 모델 정보와 체크섬 검사를 처리했습니다.
- 복원한 세션에도 새 요청의 전체 입력을 전달하고, 이전 입력과 토큰이 같은 접두부의 KV 캐시를 재사용했습니다.
- 테스트
 - iPhone 17에서 같은 5턴 대화를 캐시 끄고 켜 조건으로 각각 3회 실행해 총 30개 요청을 비교했습니다.
 - 앱을 직접 종료하고 다시 실행하며 캐시 저장과 복원 동작을 확인했습니다.
 - Mac CPU에서 서로 다른 프로세스로 파일 저장과 복원을 검증했습니다. 다음 요청과 입력 변경 조건에서, 처음부터 계산한 답변과 복원 후 생성한 답변을 비교했습니다.

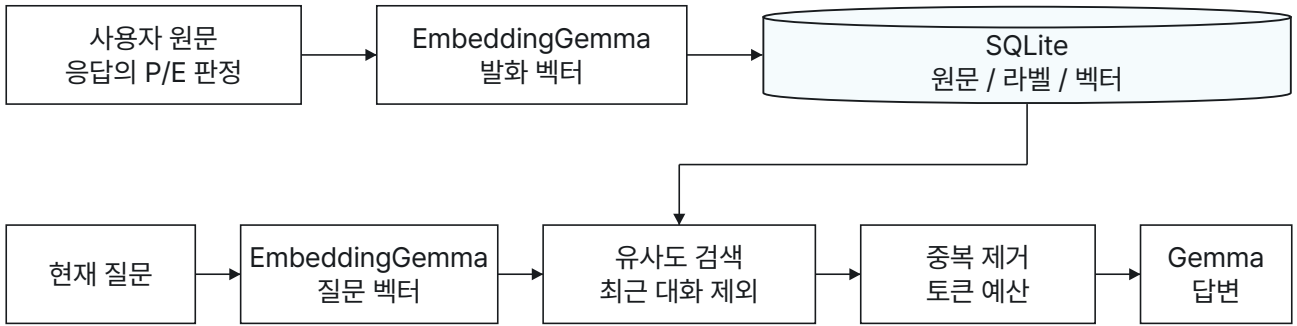
결과

메모리 KV 캐시 재사용 전후의 첫 출력 대기 시간

측정 구간	캐시 재사용 전	캐시 재사용 후
전체 평균	4.920초	1.274초
후속 요청 평균	4.872초	0.446초

- 메모리 KV 캐시 재사용으로 첫 출력 대기 시간을 전체 평균 **74.1%**, 후속 요청 평균 **90.8%** 줄였습니다. 비교한 답변 15쌍은 모두 일치했습니다.
- 파일로 저장한 CPU 세션을 다른 프로세스에서 복원한 뒤에도, 다음 요청과 입력 변경 조건의 답변이 처음부터 계산한 결과와 일치했습니다.

3. 장기 기억의 저장과 검색 구조 설계



문제

- 온디바이스 모델을 위한 전용 메모리 엔진을 설계해야 했습니다. 사전 연구에서 장문 컨텍스트 스트레스 실험을 진행했는데 전체 입력이 15.9K일 때 Gemma 4 E2B의 일상 사실 회수 정답률이 **72.0%**에서 16.9K로 늘었을 때 **22.7%**로 급락하는 **성능 저하 현상**이 관찰되었습니다.
- 이 결과를 보고 소형 모델에 맡기는 판단을 최대한 모델 외부의 결정론적 로직으로 줄여야겠다는 개발 방향성을 얻었습니다.

해결 과정

- 두 실험 결과를 보고, 검색된 기억을 가능한 모델에게 적게 전달하는 방식으로 설계를 진행했습니다.
 - 최근 대화에 이미 포함된 발화는 검색 후보에서 제외하고, 같은 문구가 중복되면 한 건만 남겼습니다.
 - 검색 결과는 기억별 토큰 길이를 확인해 입력 예산 안에서 모델에 전달했습니다.
- 장기 기억은 SQLite에 저장된 원문과 현재 발화의 임베딩 유사도를 기반으로 가져오는 것으로 구현했습니다.
 - SQLite에 발화 원문과 임베딩을 저장하고, 코드에서 ID와 시각, 사용자와 캐릭터의 기억 범위를 관리했습니다.
- 선호(P)와 사건(E) 라벨을 부여하는 판정으로 사용자의 발화에서 의미 있는 문장만 선택적으로 저장했습니다.
 - 장문 입력의 성능 저하를 보고, 개인화에 필요한 취향과 이벤트만 남겨 불필요한 발화를 걸러내려 했습니다.

결과

- PersonaMem은 과거 대화의 취향과 경험을 기억하고 질문에 답하는지 평가하는 데이터셋입니다. 같은 Gemma 4 E2B의 50문항 평가에서 기존 메모리 엔진(TencentDB Agent Memory) 적용은 **50%**, 발화 원문 검색은 **60%**였습니다. 이를 근거로 원문 검색 방식을 선택했습니다.

표 1. 기억 제공 방식별 답변 정답률 (50문항)

입력 방식	정답 수	답변 정답률
전체 대화	21/50	42%
기존 엔진 적용 (TencentDB)	25/50	50%
관련 발화 원문 검색	30/50	60%

- 기억 전달량에 따른 정답률과 추론 시간을 비교하려고, 이 중 25문항의 검색 순위를 고정하고 상위 발화를 5개, 10개, 20개씩 제공했습니다.

표 2. 전달할 발화 수 비교 (동일 25문항)

발화 수	정답 수	답변 정답률	평균 입력 (토큰)	평균 추론 (초)
5	13/25	52%	1,552	1.763
10	15/25	60%	2,488	2.160
20	15/25	60%	4,345	3.270

- 정확도는 **60%**로 같았으므로 추론 시간만 길어지는 20개 기억 설정을 쓸 이유가 없다고 판단했습니다. 그래서 최종적으로 EdgeMem의 아키텍처 설계를 확정했습니다.